BAYESIAN MACHINE LEARNING WITH
GENERATIVE AI APPLICATIONS

REALSIGHT

A Probabilistic Deepfake Detection System



Combining vision, language, and uncertainty – one posterior at a time.

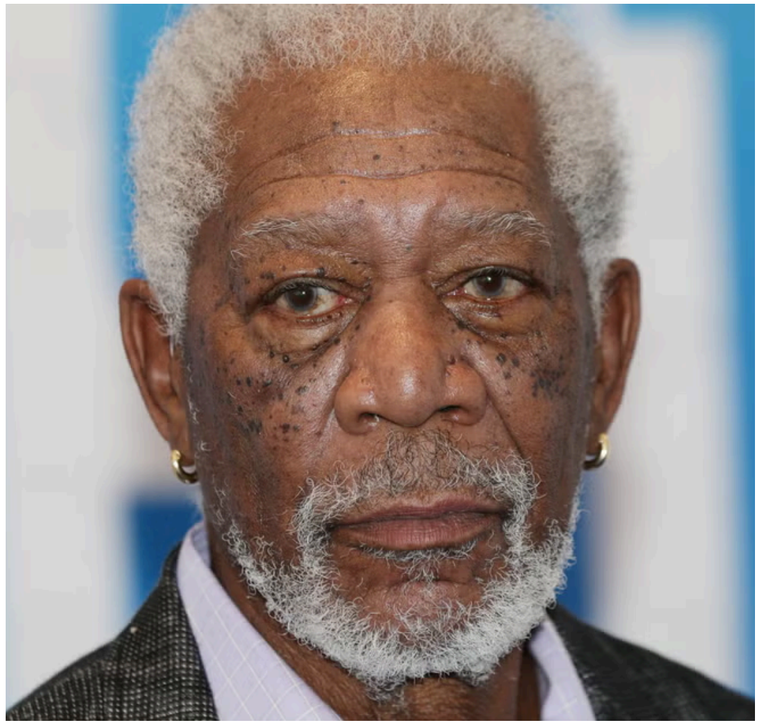
BY,
SUYOG MAHALE & ZEEL PATEL

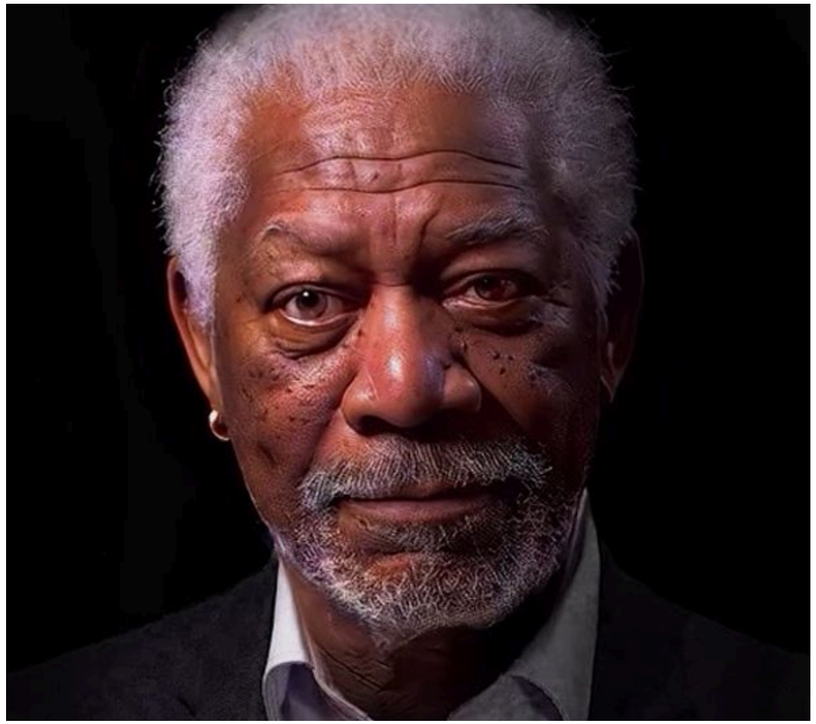
Why Build a Deepfake Detection System?

Surge in Deepfake Media

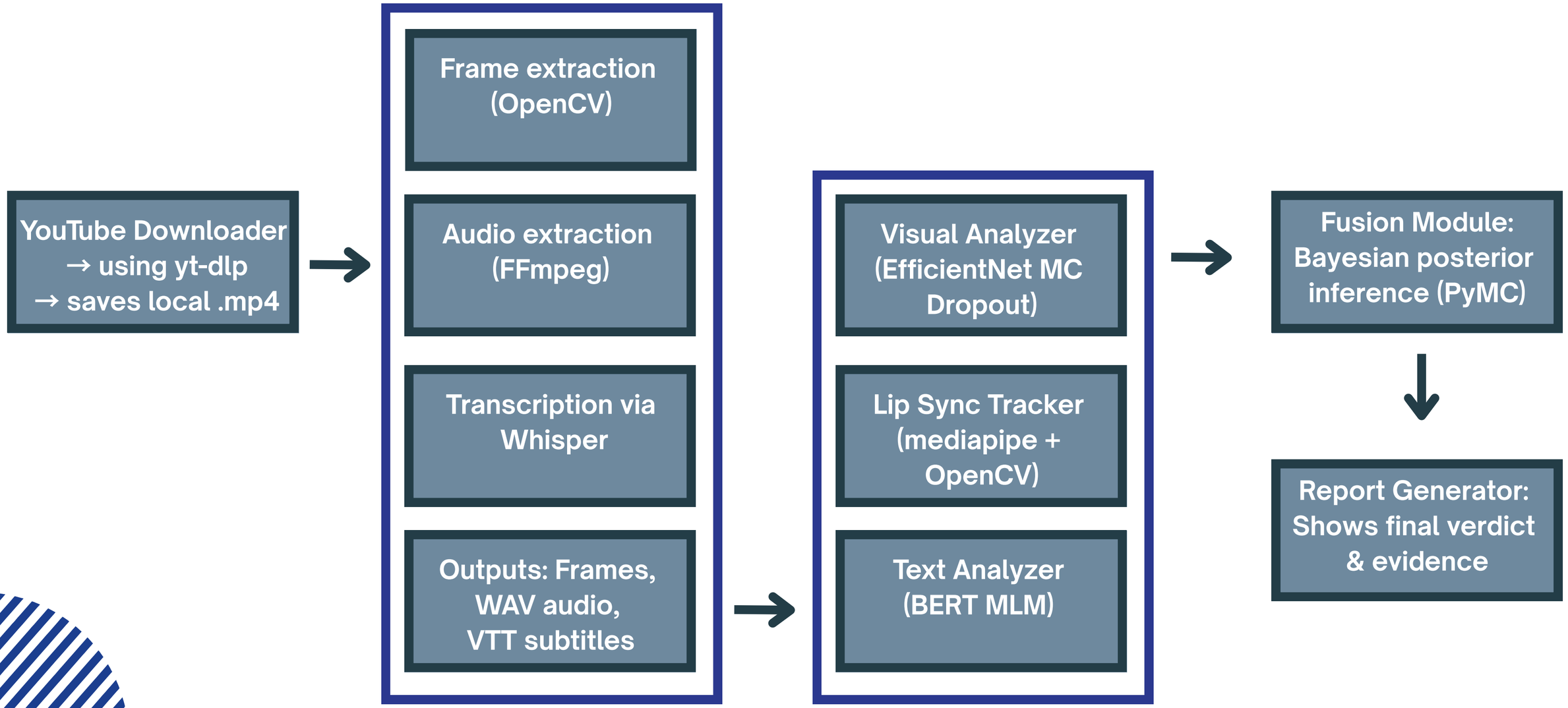
Threat to Information Integrity

Current Tools = Black Boxes

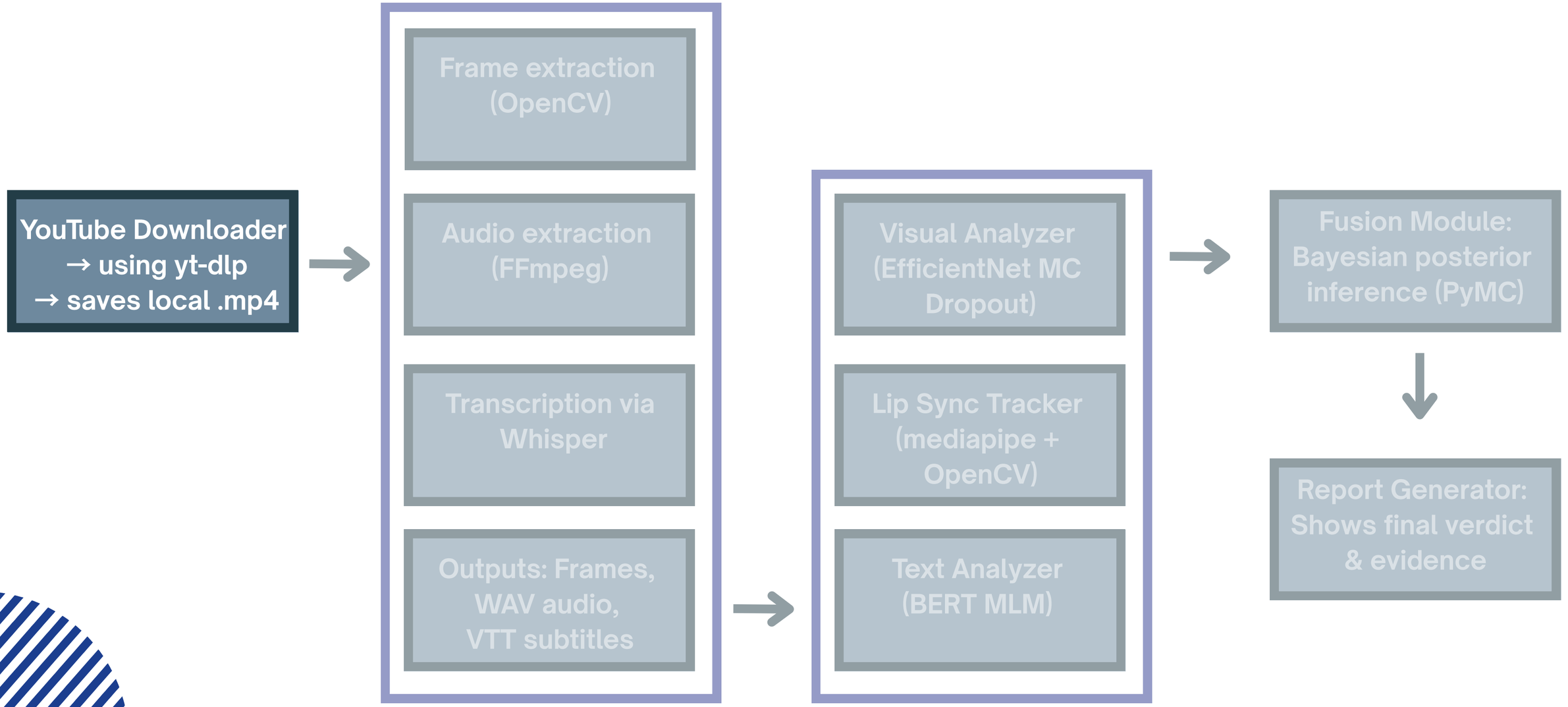




Multimodal Deepfake Detection Pipeline



Multimodal Deepfake Detection Pipeline

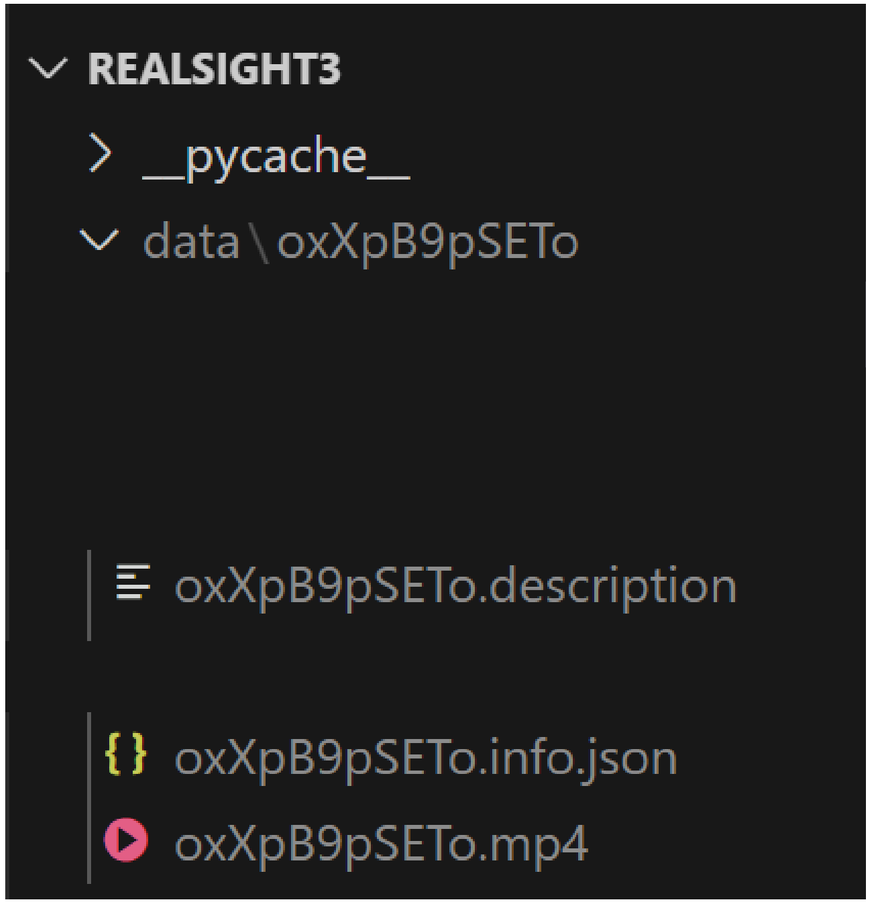


YouTube Video Downloader

```python
def download_youtube_video(url):
    print(f"Downloading video from: {url}")
    video_id = extract_video_id(url)
    if not video_id:
        print("Could not extract video ID.")
        return None

    output_path = os.path.join(DATA_DIR, video_id)
    os.makedirs(output_path, exist_ok=True)

    cmd = [
        "yt-dlp",
        url,
        "-f", "bestvideo+bestaudio/best",
        "--merge-output-format", "mp4",
        "--write-description",
        "--write-info-json",
        "--ignore-errors",
        "--no-check-certificate",
        "--force-ipv4",
        "-o", f"{output_path}/%(id)s.%(ext)s"
    ]
```



Download video → Save Subtitle → Save Metadata

YouTube Video Downloader

data > oxXpB9pSETo > oxXpB9pSETo.description

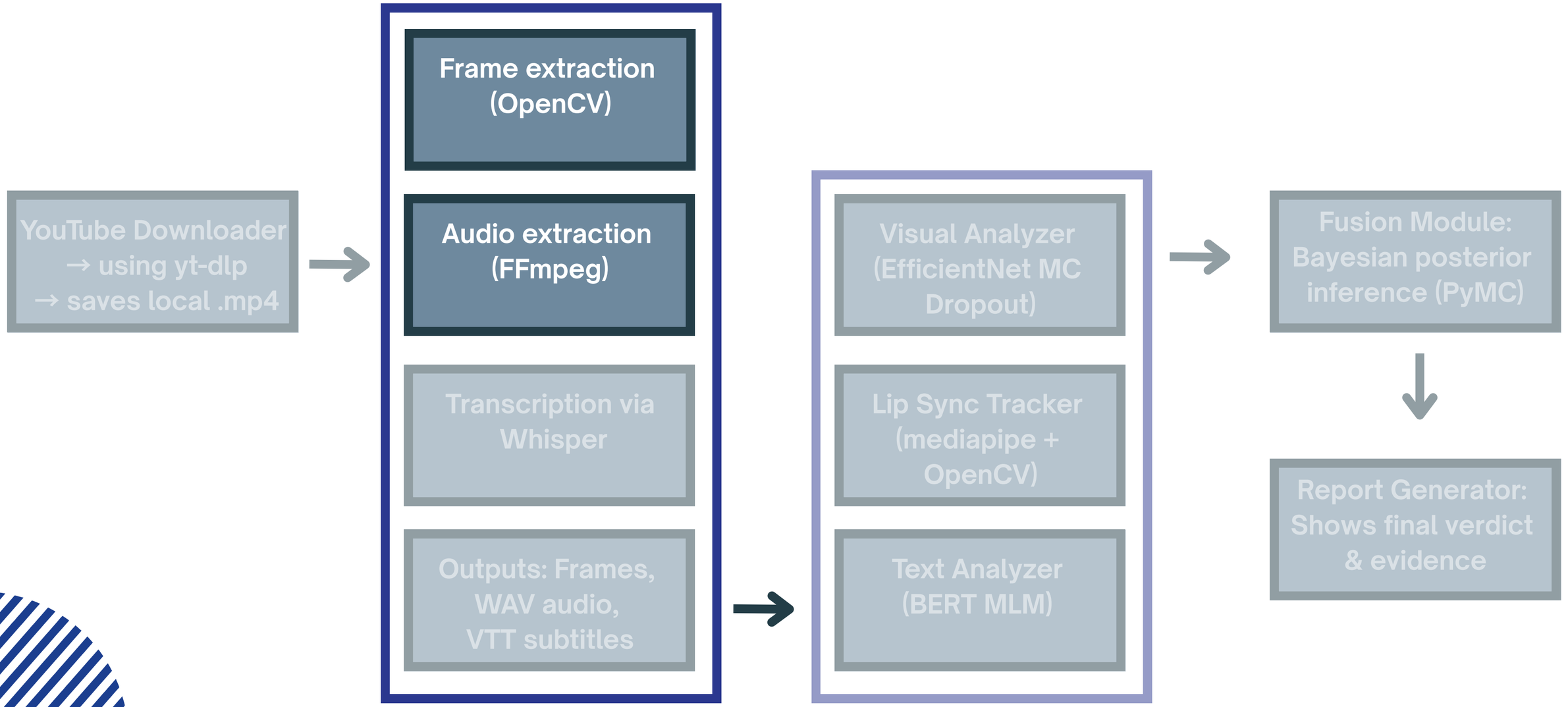
```
1  This is not Morgan Freeman  -  A Deepfake Singularity
2
3  Concept and deepfake by Bob de Jong
4  Freeman's voice is imitated by the incredibly talented Boet Schouwink
5
6  A homage to the great actor.
7
8  Contact: diepnep@gmail.com
9  #diepnep #deepfake #ai #morganfreeman
```

data > oxXpB9pSETo > {} oxXpB9pSETo.info.json > ...


```
1  {"id": "oxXpB9pSETo", "title": "This is not Morgan Freeman  -  A Deepfake Singularity", "formats": [{"format_id": "sb2",
   "format_note": "storyboard", "ext": "mhtml", "protocol": "mhtml", "acodec": "none", "vcodec": "none", "url": "https://i.
   ytimg.com/sb/oxXpB9pSETo/storyboard3_L0/default.jpg?sqp=-oaymwENSDfyq4qpAwVwAcABBqLzl_8DBgi4yba3Bg==&
   sigh=rs$AOn4CLA6AY-zNXeRpqPzNJN7_UjL8pBz9A", "width": 48, "height": 27, "fps": 1.5625, "rows": 10, "columns": 10,
   "fragments": [{"url": "https://i.ytimg.com/sb/oxXpB9pSETo/storyboard3_L0/default.jpg?
   sqp=-oaymwENSDfyq4qpAwVwAcABBqLzl_8DBgi4yba3Bg==&sigh=rs$AOn4CLA6AY-zNXeRpqPzNJN7_UjL8pBz9A", "duration": 64.0}],
   "audio_ext": "none", "video_ext": "none", "vbr": 0, "abr": 0, "resolution": "48x27", "aspect_ratio": 1.78,
   "http_headers": {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/
   97.0.4692.20 Safari/537.36", "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
   "Accept-Language": "en-us,en;q=0.5", "Sec-Fetch-Mode": "navigate"}, "format": "sb2 - 48x27 (storyboard)"}, {"format_id":
   "sb1", "format_note": "storyboard", "ext": "mhtml", "protocol": "mhtml", "acodec": "none", "vcodec": "none", "url":
   "https://i.ytimg.com/sb/oxXpB9pSETo/storyboard3_L1/M$M.jpg?sqp=-oaymwENSDfyq4qpAwVwAcABBqLzl_8DBgi4yba3Bg==&
   sigh=rs$AOn4CLBBabvL3GJKLjUPJekQy94qsM2i9g", "width": 79, "height": 45, "fps": 1.015625, "rows": 10, "columns": 10,
```




Multimodal Deepfake Detection Pipeline



Frame and Audio Extraction

```python
def extract_frames(video_path, output_folder, frame_rate=FRAME_RATE):
    os.makedirs(output_folder, exist_ok=True)

    vidcap = cv2.VideoCapture(video_path)
    fps = vidcap.get(cv2.CAP_PROP_FPS)
    if fps == 0:
        raise ValueError(f"Cannot read FPS from video: {video_path}")

    interval = int(fps / frame_rate)
    frame_id = 0
    count = 0

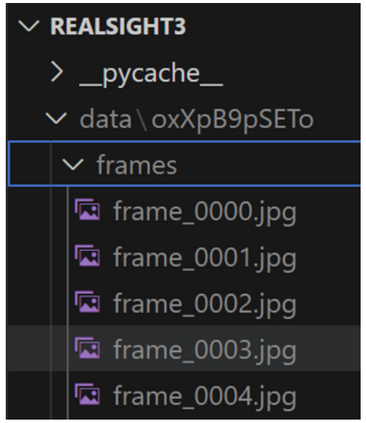
    print(f"Extracting frames from {video_path}...")

    while True:
        success, frame = vidcap.read()
        if not success:
            break

        if count % interval == 0:
            frame_path = os.path.join(output_folder, f"frame_{frame_id:04d}.jpg")
            cv2.imwrite(frame_path, frame, [int(cv2.IMWRITE_JPEG_QUALITY), 95])
            frame_id += 1

        count += 1
```

extract_frames: Slices the video into frames using OpenCV, respecting the FRAME_RATE (defined in config).



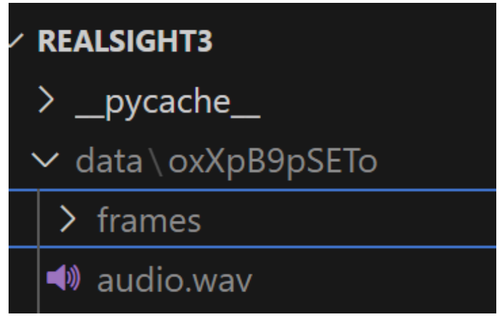

```python
def extract_audio(video_path, output_path):
    os.makedirs(os.path.dirname(output_path), exist_ok=True)

    # Check if video contains an audio stream
    if not has_audio_stream(video_path):
        raise RuntimeError(f"No audio stream found in {video_path}")

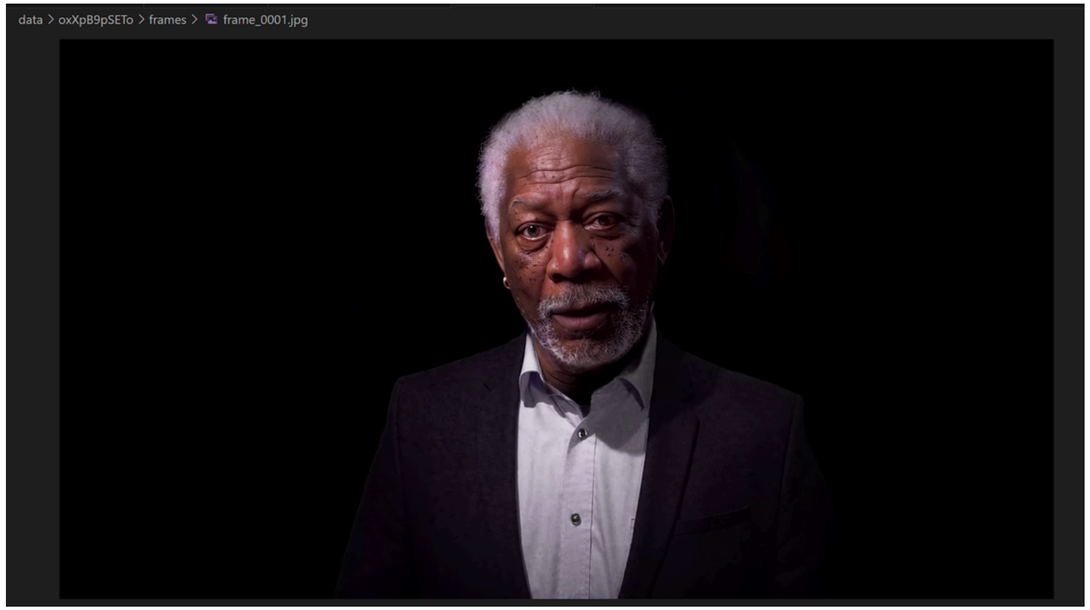
    ffmpeg_path = shutil.which("ffmpeg")
    if not ffmpeg_path:
        raise RuntimeError("ffmpeg not found in PATH")

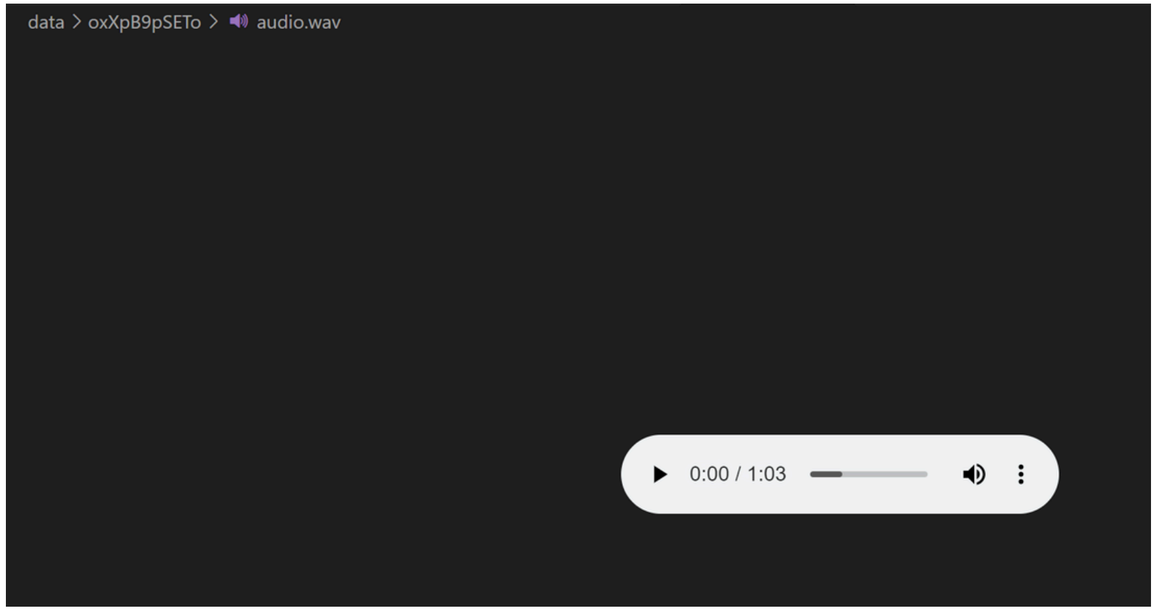
    print(f"Extracting audio using: {ffmpeg_path}")
    cmd = [
        ffmpeg_path,
        "-i", video_path,
        "-vn",
        "-acodec", "pcm_s16le",
        "-ar", "44100",
        "-ac", "2",
        output_path
    ]
```

extract_audio: Uses ffmpeg to surgically extract clean .wav audio — if the video isn't just a muted slideshow.

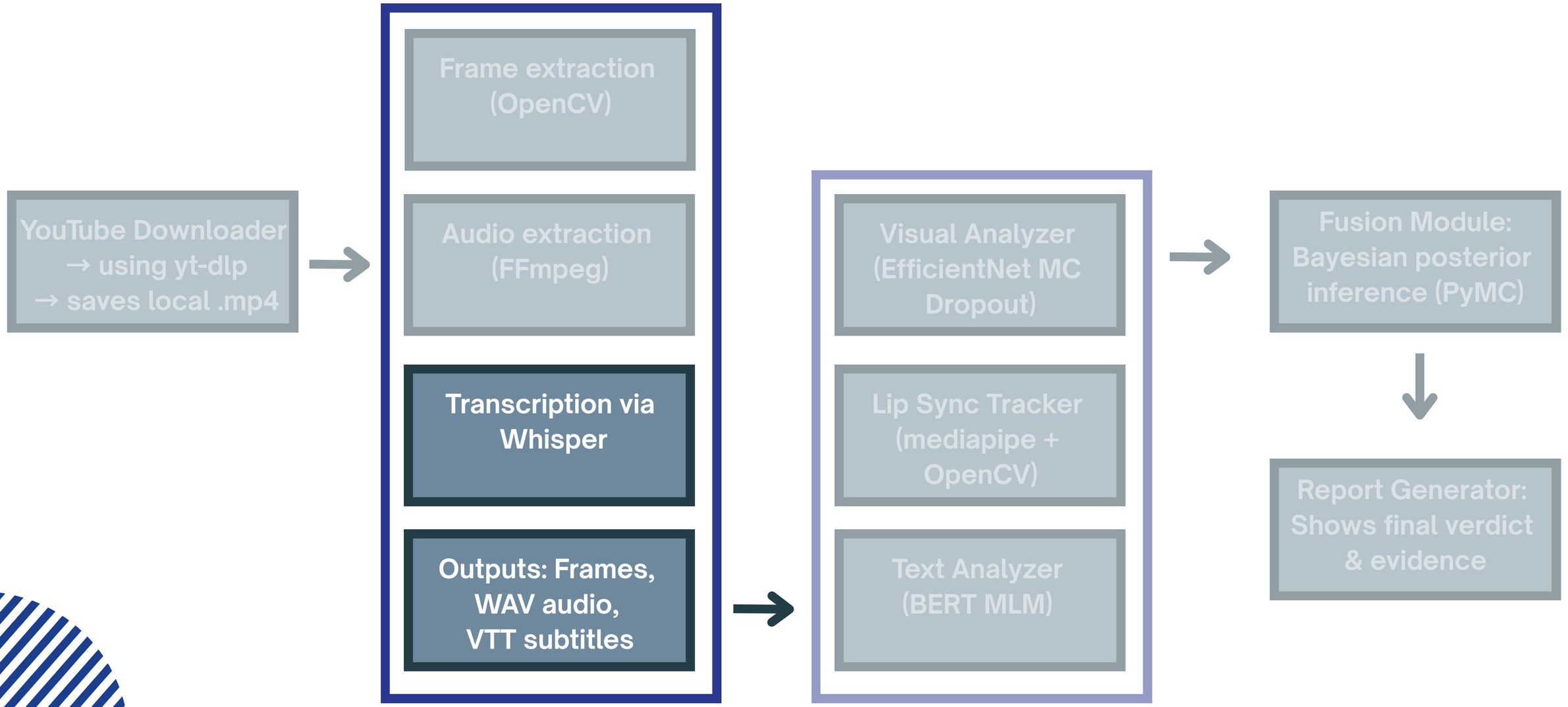


Frame and Audio Extraction





Multimodal Deepfake Detection Pipeline



Audio Transcription with Whisper

```
words = []
for segment in result.get("segments", []):
    for word in segment.get("words", []):
        words.append({
            "word": word["word"].strip(),
            "start": round(word["start"], 2),
            "end": round(word["end"], 2)
        })

output_dir = os.path.join(REPORT_DIR, video_id)
os.makedirs(output_dir, exist_ok=True)
json_path = os.path.join(output_dir, "whisper_words.json")

with open(json_path, "w", encoding="utf-8") as f:
    json.dump(words, f, indent=2)
```

Extracts word-level timestamps for precise alignment with video.

```
# Save transcript in .vtt format
def generate_vtt(segments):
    vtt_lines = ["WEBVTT\n"]
    for i, seg in enumerate(segments):
        start = format_timestamp(seg["start"])
        end = format_timestamp(seg["end"])
        text = seg["text"].strip().replace("\n", " ")
        vtt_lines.append(f"{i}\n{start} --> {end}\n{text}\n")
    return "\n".join(vtt_lines)

def format_timestamp(seconds):
    hours = int(seconds // 3600)
    minutes = int((seconds % 3600) // 60)
    secs = int(seconds % 60)
    millis = int((seconds - int(seconds)) * 1000)
    return f"{hours:02d}:{minutes:02d}:{secs:02d}.{millis:03d}"
```

Uses the Whisper model to transcribe audio into text.

Audio Transcription with Whisper

data > oxXpB9pSETo > oxXpB9pSETo.en.vtt

```
WEBVTT

0
00:00:00.000 --> 00:00:03.899
I am not Morgan Freeman and what you see is not real.

1
00:00:04.639 --> 00:00:08.359
Well, at least in contemporary terms it is not.

2
00:00:09.320 --> 00:00:12.300
What if I were to tell you that I am not even a human being?

3
00:00:13.839 --> 00:00:14.619
Would you believe me?

4
00:00:16.000 --> 00:00:17.940
What is your perception of reality?

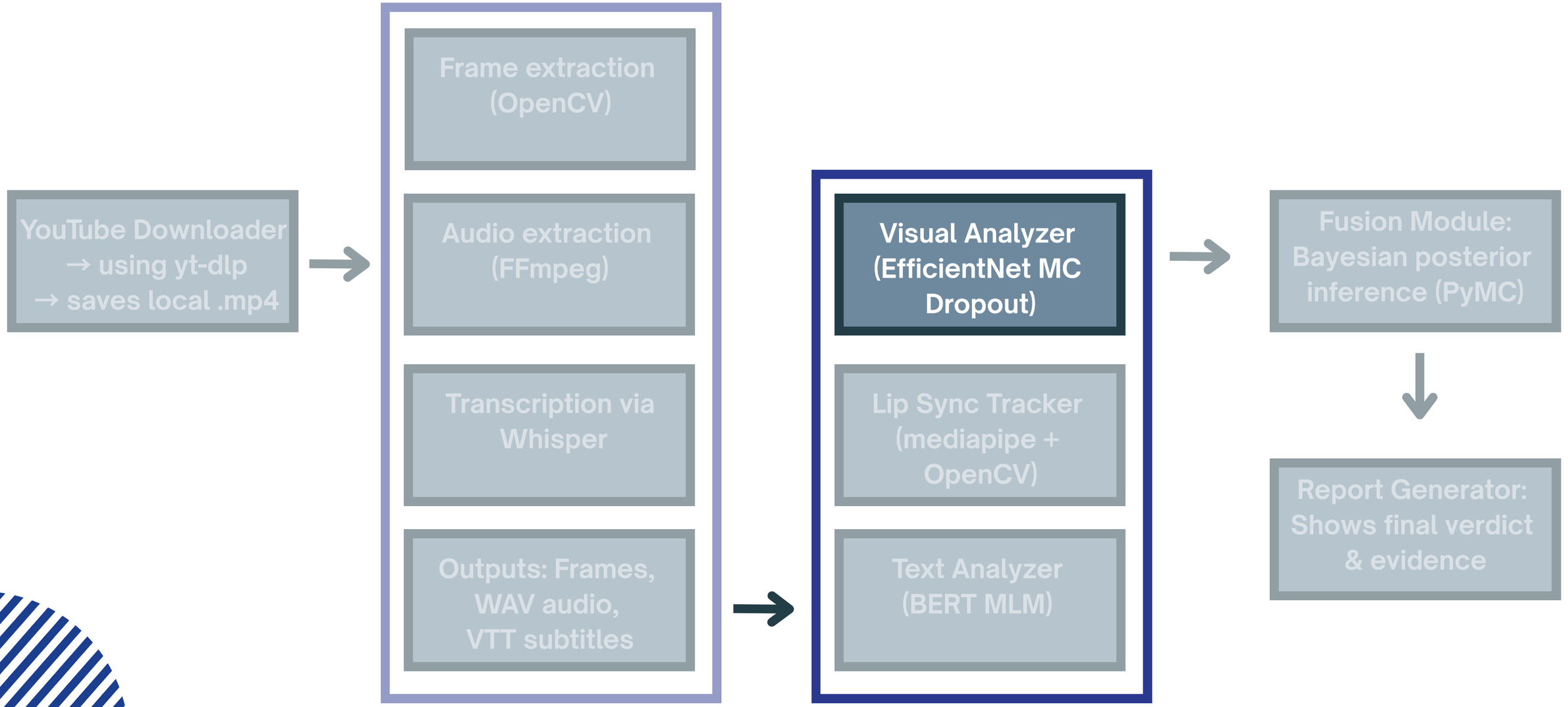
5
00:00:19.219 --> 00:00:25.559
Is it the ability to capture, process, and make sense of the information our senses receive?

6
00:00:26.439 --> 00:00:32.679
If you can see here, taste or smell something, does that make it real?
```

reports > oxXpB9pSETo > whisper_words.json > ...

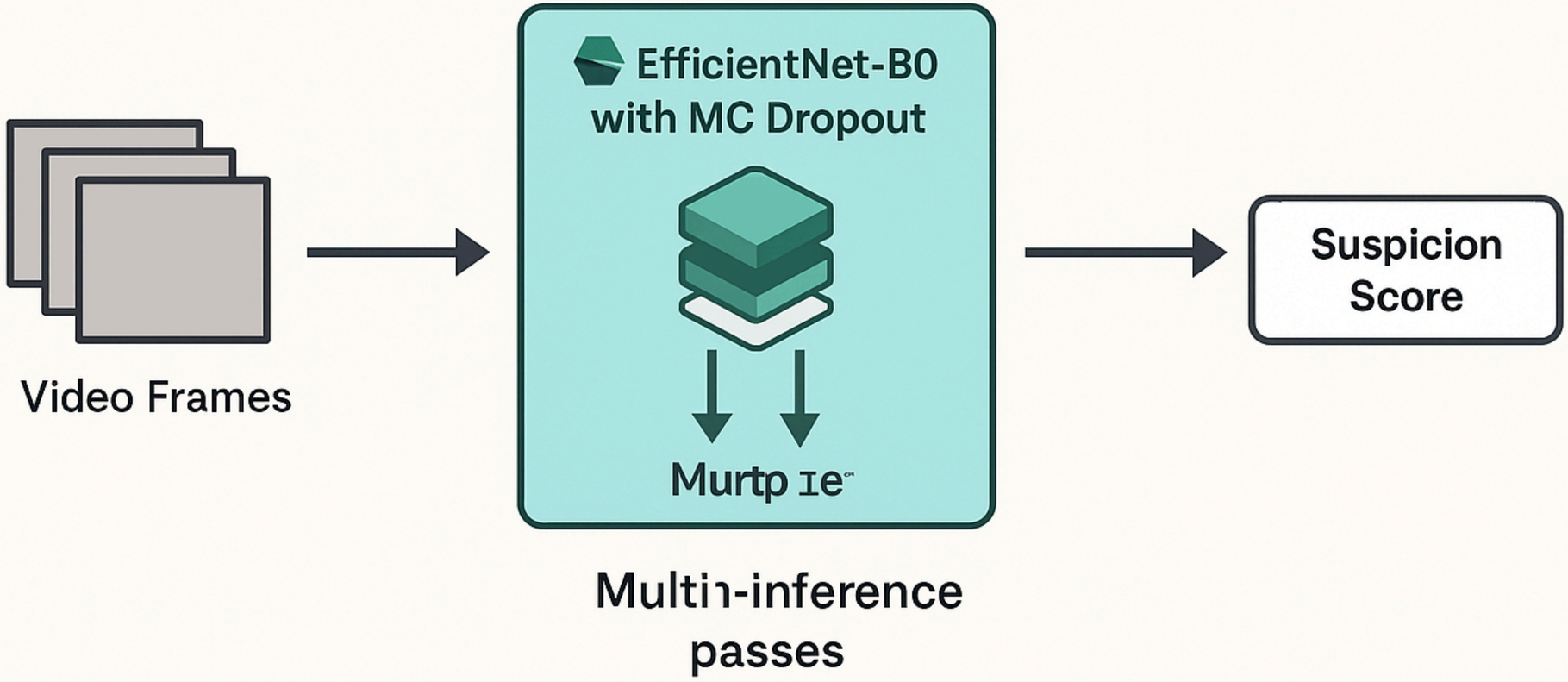

```
[
  {
    "word": "I",
    "start": 0.0,
    "end": 0.4
  },
  {
    "word": "am",
    "start": 0.4,
    "end": 0.54
  },
  {
    "word": "not",
    "start": 0.54,
    "end": 0.76
  },
  {
    "word": "Morgan",
    "start": 0.76,
    "end": 1.14
  },
  {
    "word": "Freeman",
    "start": 1.14,
    "end": 1.5
  },
  {
    "word": "and",
    "start": 1.5,
    "end": 2.32
  },
```

Multimodal Deepfake Detection Pipeline



Visual Frame Analysis using EfficientNet

Detecting Visual Anomalies via Monte Carlo Dropout



Detecting Visual Anomalies via Monte Carlo Dropout

```python
def mc_predict(model, input_tensor, n_passes=30):
    model.eval()
    enable_mc_dropout(model)
    with torch.no_grad():
        preds = [model(input_tensor) for _ in range(n_passes)]
    stacked = torch.stack(preds)
    mean_pred = stacked.mean(dim=0)
    std_pred = stacked.std(dim=0)
    return mean_pred, std_pred
```

```python
def analyze_transcript(transcript_path):
    classifier = pipeline(
        "sentiment-analysis",
        model=SENTIMENT_MODEL,
        framework="pt"
    )
```

```python
# === Sentiment Analysis ===
SENTIMENT_MODEL = "distilbert-base-uncased-finetuned-sst-2-english"
```

Detecting Visual Anomalies via Monte Carlo Dropout

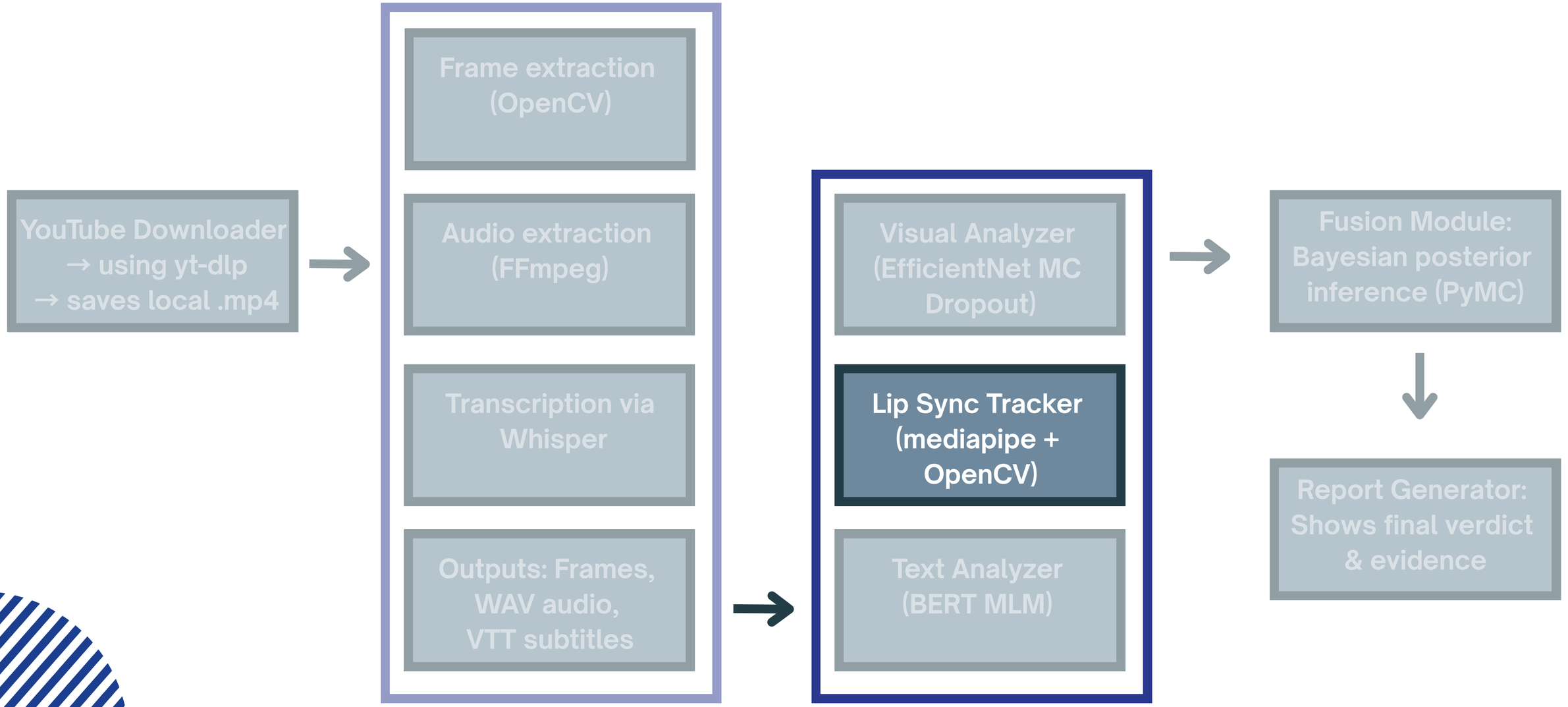
reports > oxXpB9pSETo > {} top_suspicious.json > ...


```
[
  {
    "frame": "frame_0001.jpg",
    "suspicion_score": 0.485,
    "uncertainty": 0.0809
  },
  {
    "frame": "frame_0010.jpg",
    "suspicion_score": 0.4781,
    "uncertainty": 0.1014
  },
  {
    "frame": "frame_0003.jpg",
    "suspicion_score": 0.4747,
    "uncertainty": 0.077
  },
  {
    "frame": "frame_0042.jpg",
    "suspicion_score": 0.4744,
    "uncertainty": 0.0824
  },
  {
    "frame": "frame_0062.jpg",
    "suspicion_score": 0.4638,
    "uncertainty": 0.0476
  }
]
```


reports > oxXpB9pSETo > {} sentiment.json > ...


```
{
  "average_sentiment_score": 0.795,
  "dominant_sentiment": "NEGATIVE",
  "samples_analyzed": 1
}
```

Multimodal Deepfake Detection Pipeline



Lip Sync Landmark Extraction

```python
def extract_mouth_landmarks(frames_folder, output_path):
    mp_face_mesh = mp.solutions.face_mesh
    face_mesh = mp_face_mesh.FaceMesh(static_image_mode=True)

    mouth_landmarks = {}
    print(f"Extracting mouth landmarks from: {frames_folder}")

    for file in sorted(os.listdir(frames_folder)):
        if not file.endswith(".jpg"):
            continue

        frame_path = os.path.join(frames_folder, file)
        image = cv2.imread(frame_path)
        if image is None:
            print(f"Skipping unreadable frame: {file}")
            continue

        results = face_mesh.process(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))

        if results.multi_face_landmarks:
            landmarks = results.multi_face_landmarks[0].landmark
            try:
                # Points 13 and 14 = upper and lower lip center
                top = landmarks[13]
                bottom = landmarks[14]
                mouth_landmarks[file] = [(top.x, top.y), (bottom.x, bottom.y)]
```

Library: Mediapipe Face Mesh

Approach: Process each frame, extract landmarks 13 (top lip) and 14 (bottom lip)

Output: A .npy file mapping frames to mouth coordinates

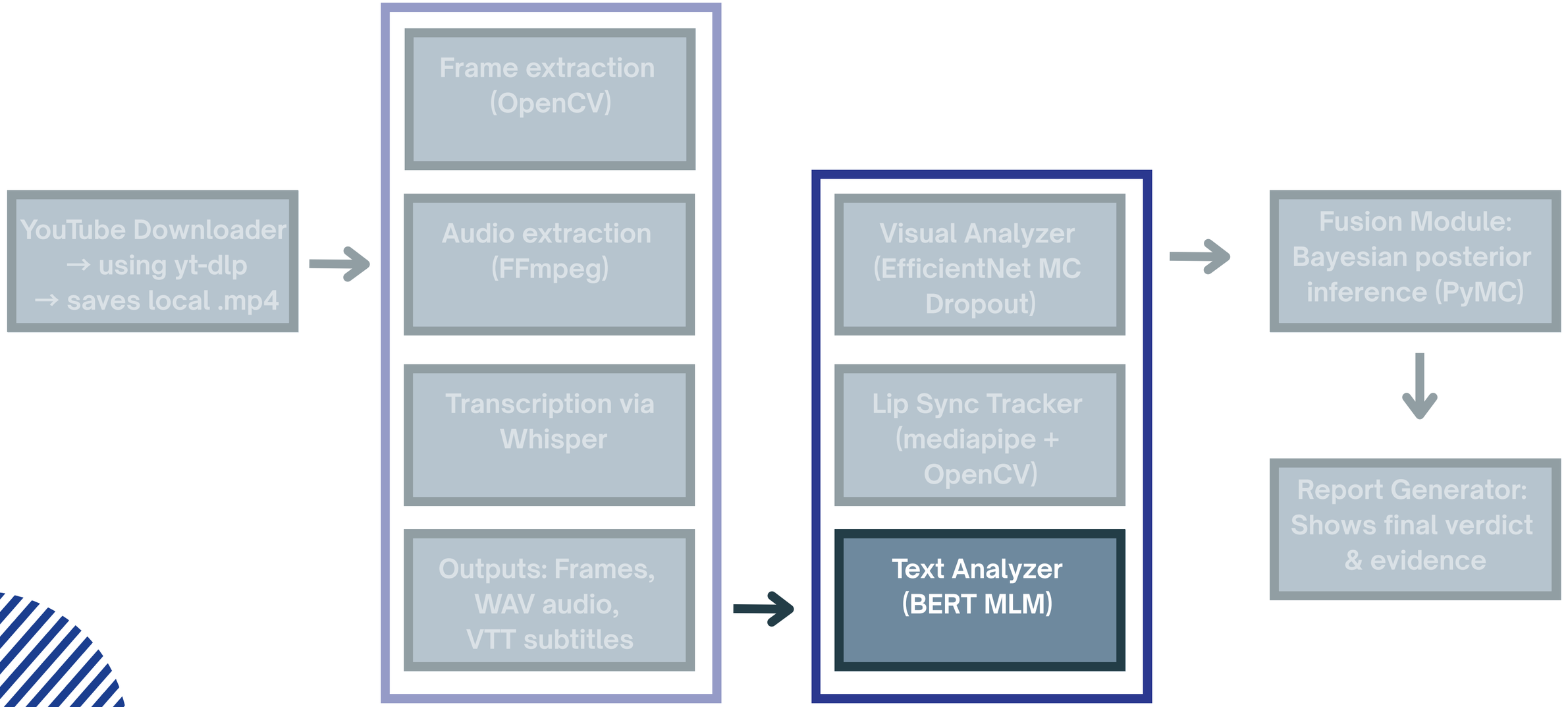
Later, this is paired with Whisper's word-level timestamps.

Lip Sync Landmark Extraction

reports > oxXpB9pSETo > {} lip_sync_analysis.json > ...


```
[
  {
    "word": "I",
    "start": 0.0,
    "end": 0.4,
    "avg_mouth_openness": 0.0106,
    "desync_flag": 1.0
  },
  {
    "word": "am",
    "start": 0.4,
    "end": 0.54,
    "avg_mouth_openness": 0.0078,
    "desync_flag": 1.0
  },
  {
    "word": "not",
    "start": 0.54,
    "end": 0.76,
    "avg_mouth_openness": 0.0064,
    "desync_flag": 1.0
  },
  {
    "word": "Morgan",
    "start": 0.76,
    "end": 1.14,
    "avg_mouth_openness": 0.0074,
    "desync_flag": 1.0
  },
```

Multimodal Deepfake Detection Pipeline



Linguistic Plausibility Check

```
text = _strip_vtt_formatting(raw_text)[:512]
model, tokenizer = _load_bert()
inputs = tokenizer(text, return_tensors="pt")
input_ids = inputs["input_ids"]

with torch.no_grad():
    outputs = model(**inputs, labels=input_ids)
    loss = outputs.loss
    perplexity = torch.exp(loss).item()

base_score = max(0.0, min(1.0, 1 / perplexity))
```

Quantify how "natural" or plausible the transcript is using masked language modeling (MLM) with BERT.

Suspicious text (e.g., AI-generated gibberish) should produce higher perplexity, indicating it's less human-like.

1. Load transcript (.vtt) → strip timestamps.
2. Pass it into bert-base-uncased using MLM-style prediction.
3. Calculate perplexity as an inverse proxy for realism.
4. Simulate uncertainty using Bayesian sampling with PyMC.

```
# Bayesian sampling
with pm.Model() as bayes_model:
    mu = pm.Normal("mu", mu=base_score, sigma=0.05)
    p_fake = pm.Deterministic("p_fake", 1 - mu)
    idata = pm.sample(2000, tune=500, chains=4, target_accept=0.9, return_inferencedata=True, progressbar=False)

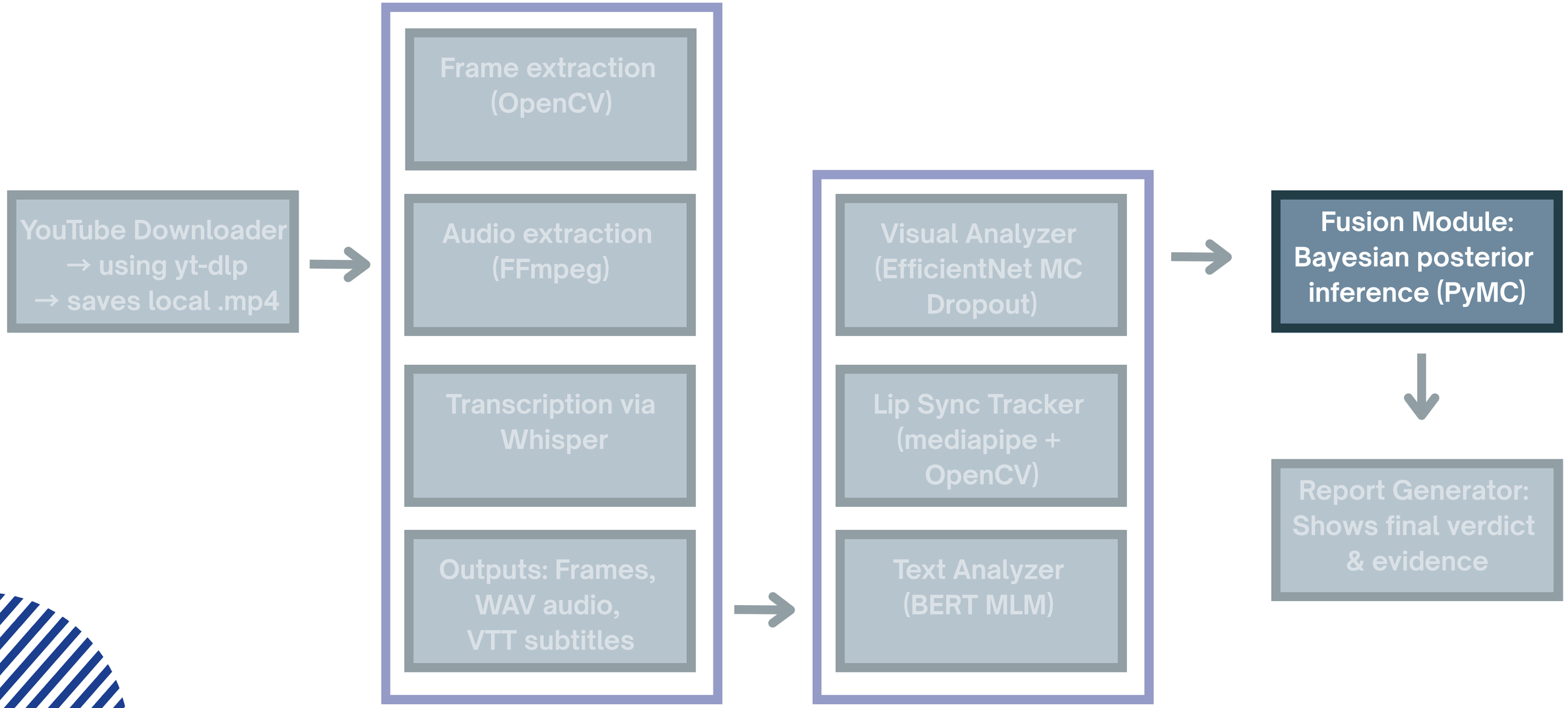
# Extract HDI from ArviZ
hdi_data = az.hdi(idata, hdi_prob=0.95)["p_fake"]
hdi_low = round(hdi_data.sel(hdi=hdi_data.coords["hdi"][0]).item(), 4)
hdi_high = round(hdi_data.sel(hdi=hdi_data.coords["hdi"][1]).item(), 4)
posterior_mean = round(idata.posterior["p_fake"].mean().item(), 4)
```

Linguistic Plausibility Check

reports > oxXpB9pSETo > {} plausibility_summary.json > ...


```
{
  "score": 0.5724,
  "hdi_2.5%": 0.4278,
  "hdi_97.5%": 0.7118,
  "perplexity": 1.39,
  "final_score": 0.6532,
  "comment": "Evaluated using BERT masked language model. Perplexity: 1.39",
  "evaluated_at": "2025-05-26T18:06:12.587733"
}
```

Multimodal Deepfake Detection Pipeline



Fusion Logic — Combining Modalities

```python
def generate_fusion_scores(mouth_landmarks_path, whisper_words_path, fps, output_path):
    mouth_landmarks = np.load(mouth_landmarks_path, allow_pickle=True).item()
    with open(whisper_words_path, "r", encoding="utf-8") as f:
        words = json.load(f)

    word_sync_scores = []
    total_openness = []
    for word_data in words:
        start, end, word = word_data["start"], word_data["end"], word_data["word"]
        start_frame = int(start * fps)
        end_frame = int(end * fps)

        openness_values = []
        for i in range(start_frame, end_frame + 1):
            frame_key = f"frame_{i:04d}.jpg"
            if frame_key in mouth_landmarks:
                top, bottom = mouth_landmarks[frame_key]
                openness = np.linalg.norm(np.array(top) - np.array(bottom))
                openness_values.append(openness)

        if len(openness_values) >= MIN_WORD_FRAMES:
            avg_open = round(float(np.mean(openness_values)), 4)
            desync = 1.0 if avg_open < LIP_SYNC_OPN_THRESHOLD else 0.0
            word_sync_scores.append({
                "word": word,
                "start": start,
                "end": end,
                "avg_mouth_openness": avg_open,
                "desync_flag": desync
            })
            total_openness.append(avg_open)
```

generate_fusion_scores:

- Extracts word-level mouth openness based on facial landmarks.
- Flags words that likely fall out of sync (desync_flag).
- Produces avg_openness, a signal for lip-sync coherence.

Fusion Logic — Combining Modalities

```python
def compute_final_score(avg_suspicion, avg_openness, plausibility_score):
    """
    Combine 3 normalized inputs into a final fake score (0.0 to 1.0)
    """
    lip_score = 1.0 if avg_openness is None else (1.0 - avg_openness)
    visual = avg_suspicion or 0.0
    semantic = 1.0 - plausibility_score  # lower plausibility = more sus

    fused_score = round(
        (WEIGHT_VISUAL * visual) +
        (WEIGHT_LIPSYNC * lip_score) +
        (WEIGHT_SEMANTIC * semantic),
        4
    )

    return {
        "visual_suspicion": visual,
        "lip_sync_suspicion": lip_score,
        "semantic_implausibility": round(semantic, 4),
        "final_score": fused_score
    }
```

compute_final_score:

- Weighted combination of:
  - Visual suspicion from frame analysis
  - Lip sync score (inverse of mouth openness)
  - Semantic implausibility (1 - BERT plausibility)

- Tunable weights from config.py

Fusion Logic — Combining Modalities

```python
def bayesian_fusion(visual_obs, transcript_obs, samples=2000):

    with pm.Model() as model:
        v = pm.Normal("visual", mu=visual_obs[0], sigma=0.1)
        t = pm.Normal("transcript", mu=transcript_obs[0], sigma=0.1)
        p_fake = pm.Deterministic("p_fake", (v + t) / 2)

        idata = pm.sample(
            2000, tune=500, chains=4, target_accept=0.9, return_inferencedata=True
        )

    hdi = az.hdi(idata, hdi_prob=0.95)["p_fake"]
    hdi_low = hdi.sel(hdi=hdi.coords["hdi"][0]).item()
    hdi_high = hdi.sel(hdi=hdi.coords["hdi"][1]).item()
    posterior_mean = idata.posterior["p_fake"].mean().values.item()

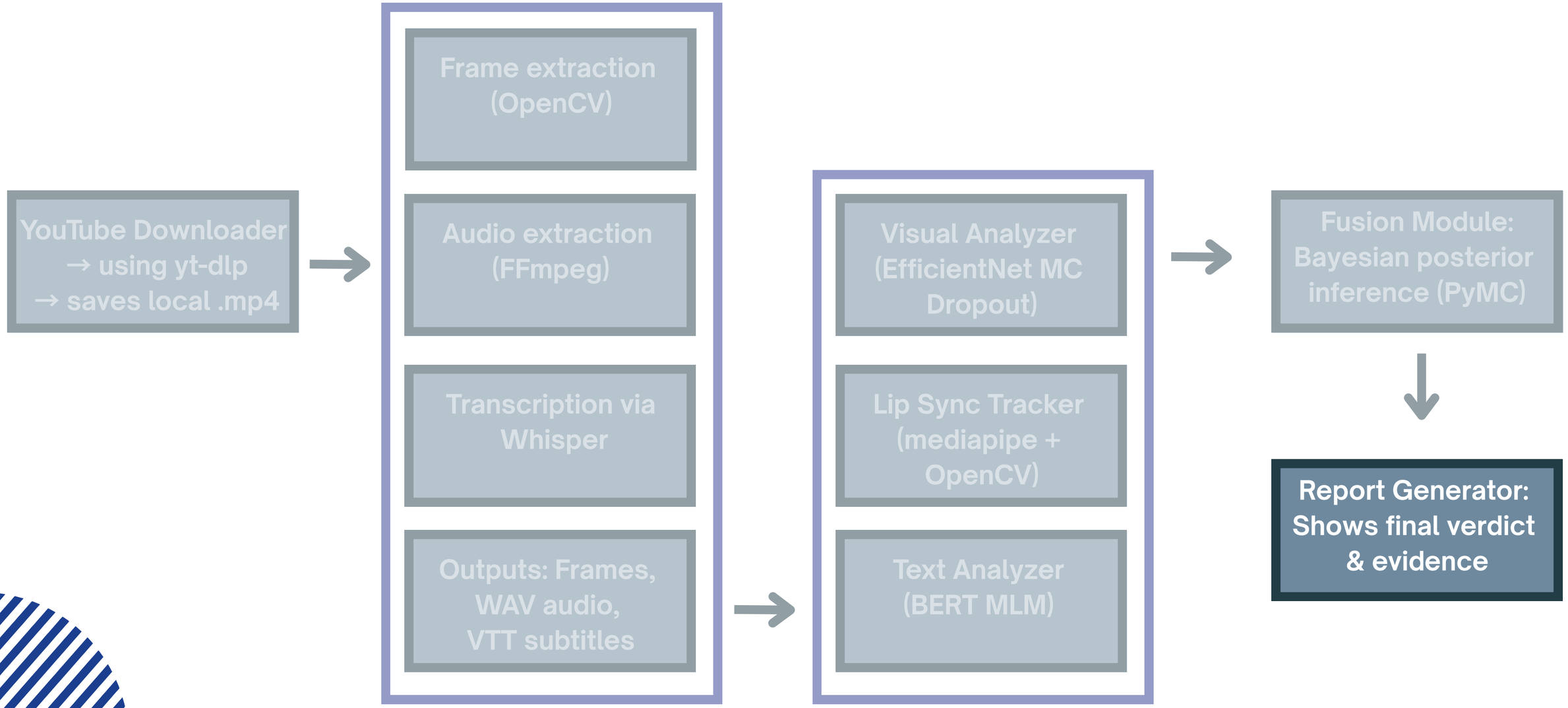
    return {
        "mean": posterior_mean,
        "hdi_3%": hdi_low,
        "hdi_97%": hdi_high,
        "idata": idata
    }
```

bayesian_fusion:

- Constructs a PyMC probabilistic model with:
  - Prior distributions from visual and transcript evidence
  - A derived variable p_fake = mean of both

- Outputs:
  - Posterior mean suspicion
  - 95% HDI bounds

Multimodal Deepfake Detection Pipeline



Packaging Outputs for Reporting

```python
# src/report.py

import os
import json
import pandas as pd
from config import REPORT_DIR

def save_all_outputs(
    video_id,
    frame_df,
    sentiment,
    lip_sync_data,
    avg_openness,
    top_suspicious,
    avg_suspicion,
    metadata,
    plausibility
):
    report_folder = os.path.join(REPORT_DIR, video_id)
    os.makedirs(report_folder, exist_ok=True)

    # Save frame analysis
    frame_data_path = os.path.join(report_folder, "visual_scores.json")
    frame_df.to_json(frame_data_path, orient="records", indent=2)

    # Save top-5 suspicious frames
    top_path = os.path.join(report_folder, "top_suspicious.json")
    with open(top_path, "w", encoding="utf-8") as f:
        json.dump(top_suspicious, f, indent=2)

    # Save sentiment
    sent_path = os.path.join(report_folder, "sentiment.json")
    with open(sent_path, "w", encoding="utf-8") as f:
        json.dump(sentiment, f, indent=2)
```

This module saves all analysis artifacts into a well-defined directory for the given video ID, making the report notebook generation modular, consistent, and reproducible.



Automatic Notebook Report Generation

```python
# notebooks/generate_report_notebook.py

import os
import json
from datetime import datetime
import nbformat as nbf
from config import REPORT_DIR, NOTEBOOK_DIR, PROJECT_NAME

def generate_notebook_report(video_id):
    report_dir = os.path.join(REPORT_DIR, video_id)
    output_path = os.path.join(NOTEBOOK_DIR, f"{video_id}.ipynb")
    os.makedirs(NOTEBOOK_DIR, exist_ok=True)

    # Load report files
    def load_json(filename):
        with open(os.path.join(report_dir, filename), "r", encoding="utf-8") as f:
            return json.load(f)

    metadata = load_json("metadata.json")
    sentiment = load_json("sentiment.json")
    lip_sync = load_json("lip_sync_summary.json")
    top5 = load_json("top_suspicious.json")
    plausibility = load_json("plausibility_summary.json")
    visual = load_json("visual_scores.json")

    # Format top 5 frame table
    table_md = "| frame | suspicion_score |\n|:------|----------------:|\n"
    for row in top5:
        table_md += f"| {row['frame']} | {row['suspicion_score']} |\n"

    avg_suspicion = round(sum([f['suspicion_score'] for f in visual]) / len(visual), 4)
    hdi_low = plausibility.get("hdi_2.5%", "N/A")
    hdi_high = plausibility.get("hdi_97.5%", "N/A")
    perplexity = plausibility.get("perplexity", "N/A")

    nb = nbf.v4.new_notebook()
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

generate_report_notebook.py

```jinja
{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "# RealSight Forensic Report\n",
    "**Generated:** {{generated_timestamp}}"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Video Metadata\n",
    "- **Video ID:** {{video_id}}\n",
    "- **Title:** {{title}}\n",
    "- **Channel:** {{channel}}"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "## Frame Analysis\n",
    "- **Average Suspicion Score:** {{avg_suspicion}}\n",
```

report_template.ipynb

Bringing It All Together - main.py

```python
# === SAVE ALL REPORT DATA BEFORE BAYESIAN FUSION ===
save_all_outputs(
    video_id=video_id,
    frame_df=frame_df,
    sentiment=sentiment_result,
    lip_sync_data=mouth_open_data,
    avg_openness=avg_openness,
    top_suspicious=top5_suspicious,
    avg_suspicion=avg_suspicion,
    metadata={"title": title, "channel": channel},
    plausibility=plausibility_result
)

# ==== BAYESIAN PROBABILISTIC ESTIMATE ====
plausibility_score = plausibility_result.get("score")
if plausibility_score is None:
    print("[NO BERT SCORE] Skipping Bayesian fusion.")
    return

result = bayesian_fusion(
    visual_obs=[avg_suspicion],
    transcript_obs=[1.0 - plausibility_score]  # flip if higher plausibility = more real
)
```

```python
# === COMPOSITE FUSION SCORE ===
final_score = compute_final_score(
    avg_suspicion=avg_suspicion,
    avg_openness=avg_openness,
    plausibility_score=result["mean"]
)

# === Update Plausibility Results with Final Bayesian Numbers ===
plausibility_result.update({
    "score": round(result["mean"], 4),
    "hdi_2.5%": round(result["hdi_3%"], 4),
    "hdi_97.5%": round(result["hdi_97%"], 4),
    "final_score": round(final_score["final_score"], 4),
})
```

```python
# === NOTEBOOK REPORT GENERATION ===
generate_notebook_report(video_id)

print(f"\n[REPORT COMPLETE] Forensic report created for video {video_id}.")
```

OUR REPORT GENERATED!

RealSight Forensic Suite Report

Generated: 2025-05-27 18:07:15

Video Metadata

- **Video ID:** oxXpB9pSETo
- **Title:** This is not Morgan Freeman - A Deepfake Singularity
- **Channel:** Diep Nep

Frame Analysis

- **Average Suspicion Score:** 0.4777
- **Top 5 Suspicious Frames:**

frame	suspicion_score
frame_0733.jpg	0.4999
frame_0744.jpg	0.4999
frame_0687.jpg	0.4998
frame_0598.jpg	0.4998
frame_0765.jpg	0.4998

Transcript Sentiment Analysis

- **Dominant Sentiment:** NEGATIVE
- **Average Score:** 0.795
- **Samples Analyzed:** 1

Lip Sync Analysis

- **Words Analyzed for Sync:** 62
- **Avg Mouth Openness (sync proxy):** 0.0089

Plausibility Check (BERT-based)

- **Plausibility Score (mean):** 0.6002
- **95% HDI Interval:** 0.4594 → 0.7373
- **Model Note:** Evaluated using a masked language model (BERT) to estimate linguistic plausibility based on average token-level perplexity.

Lower perplexity = more natural language.

- **Perplexity Estimate:** 1.39
- **Comment:**

> Evaluated using BERT masked language model. Perplexity: 1.39

Preliminary Verdict

This forensic analysis achieved a composite suspicion score of **0.6597**, based on fused Bayesian reasoning across visual, audio, and linguistic cues.

The linguistic plausibility was assessed using a BERT-based model, yielding a perplexity of **1.39** and a plausibility probability of **0.6002** (95% HDI: 0.4594 – 0.7373).

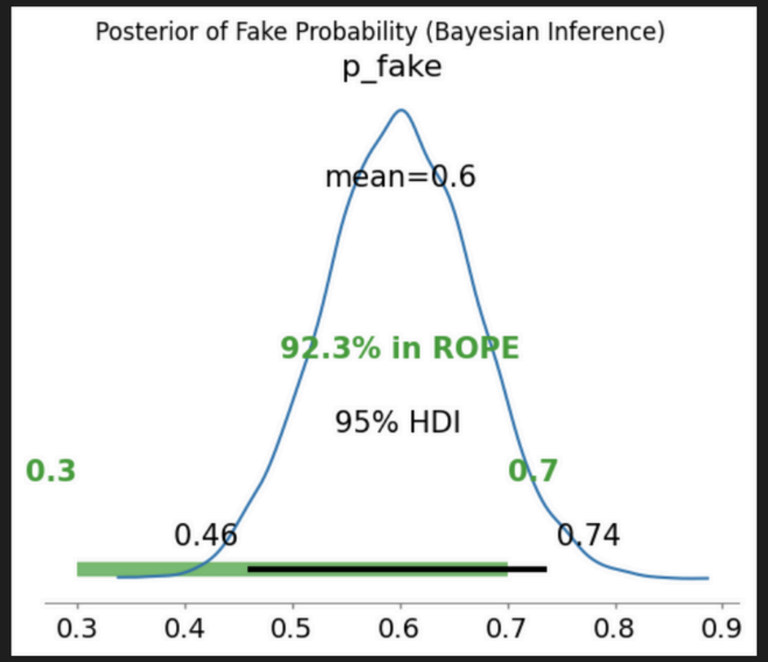
Conclusion: Moderate evidence of manipulation.

OUR REPORT GENERATED!

```python
# Posterior Distribution of Fake Probability
import matplotlib.pyplot as plt
import numpy as np
import arviz as az
import os

video_id = "oxXpB9pSETo"
report_path = os.path.join("..", "..", "reports", video_id, "inference_data.json")
idata = az.from_json(report_path)
az.plot_posterior(idata, var_names=['p_fake'], hdi_prob=0.95, rope=[0.3, 0.7])
plt.suptitle("Posterior of Fake Probability (Bayesian Inference)")
plt.show()
```

[3] ✓ 0.1s



File Structure

```
REALSIGHT3/
│
├── main.py                      ← 🔁 The brain. One script to run it all.
├── config.py                    ← ⚙️ Global constants + paths
├── requirements.txt             ← 📦 Dependencies list
├── README.md                    ← 🟦 Project overview
│
├── data/                        ← 🎥 Raw data dump (video, audio, frames)
│   └── <video_id>/
│       ├── frames/              ← Extracted video frames
│       ├── audio.wav            ← Extracted audio
│       └── *.mp4 / *.json       ← yt-dlp downloaded files
│
├── reports/                     ← 📊 Output JSONs (scores, metadata, etc.)
│   └── <video_id>/
│       ├── inference_data.json  ← Bayesian inference output
│       └── plausibility_summary.json + more
│
├── notebooks/                   ← 🔳 Final auto-generated Jupyter reports
│   ├── generate_report_notebook.py
│   └── reports/<video_id>.ipynb
│
├── templates/                   ← 📄 Notebook template (.ipynb)
│
├── src/                         ← 🧠 Core pipeline modules
│   ├── downloader.py            ← yt-dlp wrapper
│   ├── extractor.py             ← OpenCV + ffmpeg
│   ├── whisper_transcriber.py   ← Whisper + word-level timestamping
│   ├── track_mouth.py           ← MediaPipe lip landmarks
│   ├── analyzer.py              ← Xception (image) + BERT sentiment
│   ├── plausibility_checker.py  ← BERT + Bayesian posterior for language
│   ├── fusion.py                ← Bayesian fusion logic
│   └── report.py                ← Saves JSON + structured outputs
│
└── src/models/
    └── xception_loader.py       ← Loads the visual deepfake detector
```

HOW TO RUN THE WHOLE PROJECT:

Just run:
python main.py

That’s it. Everything else is automated — from downloading the video to generating a Jupyter report with Bayesian plots.


THANK YOU
Github Repository_